UM/SJTU Joint Institute
Honors Mathematics III
(VV285)

Term Project

Project Option 3
π = 243F6A8885A308D31 . . .
The Borwein-Bailey-Plouffe (BBP) formula

Name:	Duan Zhongqian	ID:	518370910031
Name:	Wu Penghao	ID:	518021910138
Name:	Liu Peijia	ID:	518370910083
Name:	Yu Yourun	ID:	518370910066
Name:	Zhou Ruixing	ID:	516370910130

Group: 23

Date: August 1, 2019

Contents

1 Abstract **3**

2 The proof of the BBP equation[2] **4**

3 PSLQ Algorithm **4**
3.1 Integer relation 4
3.2 Integer relation algorithm 4
3.3 General idea of PSLQ algorithm[9] 5
3.4 Set up (LQ factorization) 5
3.5 Hermite reduction 6
3.6 The implement of the algorithm 6
3.7 Apply PSLQ to BBP 7

4 Fundermental theories to implement the BBP algorithm **7**
4.1 Hexadecimal digit 8
4.2 Modulo calculations 9
4.3 The Binary Exponentiation Algorithm 9
4.4 The function {x} 11
4.5 Discussion of the reliability of computer calculations 11

5 The implement of the BBP algorithm **11**

6 Biography **14**
6.1 Jonathan Michael Borwein 14
6.2 Peter Benjamin Borwein 15
6.3 David Harold Bailey 15
6.4 Simon Plouffe and His Story 15

7 Conclusion and Discussion **16**

8 Reference **17**

A Code in C++ to implement the BBP algorithm **18**

1 Abstract

π, which is the ratio of the perimeter and the diameter of a circle, has been a fascinating number for millenia.

One method of obtaining the accurate number of the nth decimal digit of π is by using the Borwein-Bailey-Ploffe (BBP) formula, which is

$$\pi = \sum_{k=0}^{\infty} \frac{1}{16^k}(\frac{4}{8k+1} - \frac{2}{8k+4} - \frac{1}{8k+5} - \frac{1}{8k+6})$$

What is amazing about this formula is that it can calculate the digit at any position of π without finding digits before it.

In our project, we first prove the correctness of the BBP formula. Then, introduction of a very vital algorithm PSLQ is included in the project. The following part will use the BBP formula to calculate the nth hexadecimal digit fo π without calculating the previous digit. Finally, we use knowledge of programming to realize the algorithm.

2 The proof of the BBP equation[2]

For any $p < 8$,

$$\int_0^{\frac{1}{\sqrt{2}}} \frac{x^{p-1}}{1-x^8}dx = \int_0^{\frac{1}{\sqrt{2}}} \sum_{i=0}^{\infty} x^{p-1+8k}dx = \sum_{i=0}^{\infty} \frac{1}{k+8p} x^{p+8k}|_0^{\frac{1}{\sqrt{2}}} = \frac{1}{2^{\frac{p}{2}}} \sum_{k=0}^{\infty} \frac{1}{16^k(8k+p)} \tag{1}$$

Then, we can get

$$\sum_{k=0}^{\infty} \frac{1}{16^k(8k+p)} = 2^{\frac{p}{2}} \int_0^{\frac{1}{\sqrt{2}}} \frac{x^{p-1}}{1-x^8}dx \tag{2}$$

Therefore, the right hand side of the equation can be written as

$$\sum_{k=0}^{\infty} \frac{1}{16^k}(\frac{4}{8k+1} - \frac{2}{8k+4} - \frac{1}{8k+5} - \frac{1}{8k+6}) = \int_0^{\frac{1}{\sqrt{2}}} \frac{4\sqrt{2} - 8x^3 - 4\sqrt{2}x^4 - 8x^5}{1-x^8}dx \tag{3}$$

Substitute $y = \sqrt{2}x$, we get

$$\int_0^1 \frac{16y-16}{y^4-2y^3+4y-4}dy = \int_0^1 \frac{16y-16}{(y^2-2)(y^2-2y+2)}dy = \int_0^1 \frac{4y}{y^2-2}dy - \int_0^1 \frac{4y-8}{y^2-2y+2}dy \tag{4}$$

$$= 2ln(-1) - 2ln(-2) + 2ln(2) - 4tan^{-1}(-1) = \pi$$

3 PSLQ Algorithm

3.1 Integer relation

For a vector x in $\mathbb{R}^n$ or $\mathbb{C}^n = \mathbf{x} = (x_1, x_2, \cdots, x_n)$, if there exists a vector $m = (m_1, m_2, \cdots, m_n)$ which consists of n integers such that

$$m_1x_1 + m_2x_2 + \cdots m_nx_n = 0$$

then we say that x has an integer relation, if m_i are not all zero.
For example, a vector $x = (1, x, x^2, x^3, \cdots, x^{16})$ where $x = 3^{1/4} - 2^{1/4}$ has an integer relation with $m = (1, 0, 0, 0, -3860, 0, 0, 0, -666, 0, 0, 0, -20, 0, 0, 0, 1)$, such that $1 - 3860x^4 - 666x^8 - 20x^{12} + x^{16} = 0$.
And this is just a case of finding the polynomial of degree $n = rs$ with $x = 3^{1/r} - 2^{1/s}$[12].

3.2 Integer relation algorithm

Integer relation algorithm is a computational method to find whether the vector exists or produce bounds of the norm of all possible relation, where **norm of integer**

relation denotes the Euclidean norm of m[6].
There are many algorithms to detect the integer relations such as LLL algorithm, PSOS algorithm HJLS algorithm, Freguson-Forcade algorithm and PSLQ algorithm. PSLQ algorithm is one of the most widely used algorithm with outstanding performance and excellent numerical stability to study the integer relation.
In the next part, the algorithm will be discussed in detail and some applications will be given.

3.3 General idea of PSLQ algorithm[9]

PSLQ is one of the effective algorithms of integer relation detection. It is found by Ferguson and its name comes from "its usage of a partial sum-of-squares vector and an LQ (Lower-diagonal-orthogonal) matrix factorization" [7]. The detailed implement of PSLQ algorithm will be discussed below.

3.4 Set up (LQ factorization)

Let the vector of real numbers $\mathbf{x} = (x_1, x_2, \cdots x_n)$ with $\mid \mathbf{x} \mid = 1$. Then a n$\times$ (n-1) matrix H_x is defined as :
Definition 1:

$$H_x = \begin{Bmatrix} \frac{s_2}{s_1} & & & \\ -\frac{x_1x_2}{s_1s_2} & \frac{s_3}{s_2} & & \\ -\frac{x_1x_3}{s_1s_2} & -\frac{x_2x_3}{s_2s_3} & \ddots & \\ \vdots & \vdots & \ddots & \frac{s_n}{s_{n-1}} \\ -\frac{x_1x_n}{s_1s_2} & -\frac{x_2x_n}{s_2s_3} & \cdots & -\frac{x_{n-1}x_n}{s_{n-1}s_n} \end{Bmatrix}$$

Another form to express H_x is $H_x = (h_{i,j})$ and

$$h_{i,j} = \begin{cases} 0 & if\ 1 \le i < j \le n-1 \\ s_{i+1}/s_i & if\ 1 \le i = j \le n-1 \\ -x_i^2/(s_js_{j+1}) & if\ 1 \le j < i \le n \end{cases}$$

where the partial sums s^2 are defined
Definition 2:

$$s^2 := \sum_{j=k}^{n} x_j^2.$$

Then the (n-1) columns of H_x are orthogonal so they together with x form a orthogonal basis for the relations of x.
Proof.
We first need to show columns of H_x are orthogonal. Considering two columns h_i and h_j
When $i < j$, the inner product is

$$-\frac{s_{j+1}x_ix_j}{s_js_is_{i+1}} + \frac{x_ix_j}{s_is_{i+1}s_js_{j+1}} \sum_{j<k\le n} x_k^2 = 0$$

When $i = j$ the inner product is

$$\frac{s_{i+1}^2}{s_i^2} + \sum_{i<k\leq n} \frac{x_i^2 x_k^2}{s_i^2 s_{i+1}^2} = \frac{s_{i+1}^2}{s_i^2} + \frac{x_i^2}{s_i^2 s_{i+1}^2} \sum_{i<k\leq n} x_k^2 = \frac{s_i^2 - x_i^2}{s_i^2} + \frac{x_i^2}{s_i^2} = 1$$

Then we need to prove $xH_x = 0$, fix $1 \leq j \leq n-1$ we have

$$\sum_{1\leq k\leq n} x_k h_{k,j} = \frac{x_j s_{j+1}}{s_j} - \sum_{j<k\leq n} \frac{x_k^2 x_j}{s_j s_{j+1}} = \frac{x_j s_{j+1}}{s_j} - \frac{x_j s_{j+1}^2}{s_j s_{j+1}} = 0$$

For x $\in \mathbb{R}^n$, let A be an invertible matrix of integers, if Q is an orthogonal matrix such that $AH_XQ = L$ is lower trapezoidal, we have

Theorem

$$\mid m \mid \geq \frac{1}{\mid maxL_{jj} \mid}$$

for any integer relation m. The basic idea of the PSLQ algorithm is actually find A and Q iteratively to find the lower bound for a certain precision.

3.5 Hermite reduction

Let H be a lower trapezoidal matrix with $h_{i,i} \neq 0$. Then an integer matrix $D = (d_{i,j})$ is defined recursively as below

$$d_{i,j} = \begin{cases} 0 & if\ i < j \\ 1 & if\ i = j \\ [-\frac{1}{h_{jj}} \sum_{k=j+1}^{i} D_{ik} H_k j] & if\ i > j \end{cases}$$

where $[x] = \lfloor x + 1/2 \rfloor$.

The matrix D is the Hermite reducing matrix of H and DH is the Hermite reduction of H. The PSLQ algorithm is continuously find D and replace A with DA.

3.6 The implement of the algorithm

Initial conditions

Fix a real number $\gamma > 2/\sqrt{3}$. H_x is the matrix in definition 1, and let matrix A be the Hermite reducing matrix of H_x. Matrix I is the identity matrix.

Exchange

Find the j such that $\gamma^j |(AH_xQ)_{jj}|$ reaches the maximum and then set $r = j$. If $r = n-1$, go to the step reduction. Exchange the rth row with the $r+1$th row of matrix A.

Corner

Define$\alpha = (AH_xI)_{r,r}$, $\beta = (AH_xI)_{r,r+1}$, $\lambda = \sqrt{\alpha^2 + \beta^2}$.
Let Q a (n-1) $\times$ (n-1) orthogonal matrix defined as

$$q_{i,j} = \begin{cases} \alpha/\lambda & if\ i = j = r\ or\ i = j = r+1 \\ \beta/\lambda & if\ i = r+1, j = r \\ -\beta/\lambda & if\ i = r, j = r+1 \\ 1 & if\ i = j \neq r\ or\ i = j \neq r+1 \\ 0 & otherwise \end{cases}$$

Replace I by IQ.

Reduction
Let D be the Hermite reducing matrix of AH_xI. Replace A by DA.

Termination
By the step Corner, the lower bound$|\ m\ | \geq 1/max_j((AH_xI)_{jj})$ is established.
Now, if all components of xA^{-1} is non-zero, go back to step Exchange.
If xA^{-1} has a zero component, the integer relation m is found as the corresponding column of the integer matrix A^{-1} .

Note
Different γ should be chosen for different precision needs.

3.7 Apply PSLQ to BBP

The BBP formula can be found by applying PSLQ. Generally speaking, PSLQ is used to find the integer relation between pi and the constants $X_j = \sum_{k=0}^{\infty} \frac{1}{16^k(8k+j)}$ where i = 1,2,3,$\cdots$,8. This allows us computes the nth hexadecimal digit without computing other $n-1$ digits before with little computer memory.
What's more, the PSLQ also found the base 3(ternary) formula for π^2.

$$\pi^2 = \frac{2}{27}\sum_{k=0}^{\infty}\frac{1}{729^k}\Big(\frac{243}{(12k+1)^2} - \frac{405}{(12k+2)^2} - \frac{81}{(12k+4)^2} - \frac{27}{(12k+5)^2} - \frac{72}{(12k+6)^2} - \frac{9}{(12k+7)^2} - \frac{9}{(12k+8)^2} - \frac{5}{(12k+10)^2} + \frac{1}{(12k+11)^2}\Big)\ [7]$$

4 Fundermental theories to implement the BBP algorithm

The Borwein-Bailey-Plouffe formula, which also can be called as BBP formula, is shown as follows. BBP formula can be used to calculate an arbitrary hexadecimal digit of π, without calculating the previous digits, and the proof will been shown

later.

$$\pi = \sum_{k=0}^{\infty} \frac{1}{16^k} \left(\frac{4}{8k+1} - \frac{2}{8k+4} - \frac{1}{8k+5} - \frac{1}{8k+6} \right) \tag{5}$$

4.1 Hexadecimal digit

First of all, in mathematics and computing, hexadecimal is a positional numerical system with a base of 16, which uses sixteen distinct symbols "0"-"9" to represent zero to nine and "A"-"F" to represent ten to fifteen [10]. If a real number is written in the base of 16, it can be represented by a series of finite of infinite digits and can be transferred in the base of 10 like

$$\left(\overline{\ldots d_{i_6} \ldots d_{2_{16}} d_{1_{16}} d_{0_{16}} . d_{-1_{16}} d_{-2_{16}} d_{-3_{16}} \ldots}\right)_{16} = \sum_{i=0}^{+\infty} 16^i \cdot (d_{i_6})_{10} + \sum_{i=-1}^{-\infty} 16^i \cdot (d_{i_6})_{10}\,, \tag{6}$$

where the signs $d_i 16$ means the hexadecimal digit of the ith position in the hexadecimal numerical system (.)16, while the notation (.)10 means that the number inside is in the base of 10 (the base generally used). For instance, when a number x with fractional part and integral part smaller than 16 in decimal form (like π) is transferred to a hexadecimal form by computing, which can be shown as

$$x = \left(\overline{0.d_{1_{10}} d_{2_{10}} d_{3_{10}} \ldots}\right)_{10} = \sum_{i=-1}^{\infty} 16^i \cdot (d_{i_{16}})_{10} = \left(\overline{0.d_{1_{16}} d_{2_{16}} d_{3_{16}} \ldots}\right)_{16} \tag{7}$$

In order to transfer the number x into base of 16, we can keep multiply it by 16 and the integer part of the result will be the digit we need in base 10. Afterwards, we carry on the process by take the fractional part of the previous result and repeat the process above. After calculations, we can get the hexadecimal number by recursion, while the code is shown in the Appendix. If we have found n hexadecimal digits from $d_{0,16}$ to $d_{n-1,16}$, we can find the $(n+1)^{th}$ hexadecimal digit $d_{0,16}$ of x. When i = n and all of the hexadecimal digits from 0th to $(n-1)^{th}$ has been found,

$$x_{n-1} = \left\{16^{n-1} x\right\} = 16^{n-1} \left(x - \sum_{i=1}^{n-1} 16^{-i} \left(d_{i_{16}}\right)_{10} \right) = 16^{n-1} \left(\sum_{i=n}^{\infty} 16^{-i} \left(d_{i_{16}}\right)_{10} \right)$$

$$= \sum_{j=1}^{\infty} 16^{-j} \left(d_{j+n-1_{16}}\right)_{10} = \left(\overline{0.d_{n_{16}} d_{n+1_{16}} d_{n+2_{16}} d_{n+3_{16}} \ldots}\right)_{16} = \left(\overline{0.d_{n-1,2_{10}} d_{n-1,3_{10}} d_{n-1,4_{10}} \ldots}\right)_{10}$$

Multiply two sides by 16

$$16 \times x_{n-1} = \left(\overline{d_{n,1_{10}} . d_{n,2_{10}} d_{n,3_{10}} d_{n,4_{10}} \ldots}\right)_{10} = 16^0 d_{n,1_{10}} + \left(\overline{0.d_{n,2_{10}} d_{n,3_{10}} d_{n,4_{10}} \ldots}\right)_{10} = 16^0 d_{n,1_{10}} + \{x_n\}$$

$$= 16^0 \left(d_{n_{16}}\right)_{10} + \left(\overline{0.d_{n+1_{16}} d_{n+2_{16}} d_{n+3_{16}} \ldots}\right)_{16} = \left(\overline{d_{n_{16}}.d_{n+1_{16}} d_{n+2_{16}} d_{n+3_{16}} \ldots}\right)_{16}$$

Therefore, we have

$$\left(d_{n_{16}}\right)_{10} = d_{n,1_{10}} [1]$$

$$\text{and } x_n = \left(\overline{0.d_{n,2_{10}} d_{n,3_{10}} d_{n,4_{10}} \ldots}\right)_{10} = \left(\overline{0.d_{n+1_{16}} d_{n+2_{16}} d_{n+3_{16}} \ldots}\right)_{16}$$

$$= \{16^n x\} = 16^n \left(x - \sum_{i=1}^{n} 16^{-i} \left(d_{i_{16}}\right)_{10}\right) = \sum_{j=1}^{\infty} 16^{-j} \left(d_{j+n_{16}}\right)_{10} [2]$$

Actually, when written in code, the function that transfers a fraction number in base of 10 into base of 16 is much easier than the mathematical proof. The corresponding code is shown in Appendix as the function "string deci2hex(ld x)".

4.2 Modulo calculations

Secondly, in the proof is a numerical tools which is called modulo calculation, which can be represented by a notation as

$$a = x \bmod y \tag{8}$$

which means that a is the remainder of x divided by y, where x, y, and a are all positive integral. For instance, if we want to take the fraction part of x/y, which is noted by x/y, we can use the modulo calculation as

$$\left\{\frac{x}{y}\right\} = \left\{b + \frac{x \bmod y}{y}\right\} = \left\{\frac{x \bmod y}{y}\right\} \tag{9}$$

where b is the quotient of x divided by y, and all of the numbers are positive integrals.

4.3 The Binary Exponentiation Algorithm

Thirdly, in the proof, there is a numerical method applied called the modular exponentiation, which is a exponentiation performed over a modulus. In this case, we are going to calculate the remainder c when an integral b, which is the base, is in the power of p and divided by a, where a, b, p are all integrals, so that the equation can be shown as.

$$c = b^p \bmod a \tag{10}$$

For instance, a=b=4,p=7, if we are computing c, the calculation of bp, which is 4^7 may take too much time, especially when the base and exponent are extremely large (like over than 2^30). As a result, we can use the Binary Exponentiation Algorithm. First, the exponent p can be represented by a binary series as

$$p = \sum_{i=0}^{n-1} k_i 2^i \tag{11}$$

where the length of p is n, and ki can only take 1 or 0 and kn-1=1, so that the bp can be written as

$$b^p = b^{\left(\sum_{i=0}^{n-1} k_i 2^i\right)} = b^{\left(k_0+2k_1+2^2k_2+\cdots+2^{n-1}\right)} = \prod_{i=0}^{n-1} \left(b^{2i}\right)^k \tag{12}$$

Therefore, c can be calculated as

$$c = b^p \bmod a = \prod_{i=0}^{n-1} \left(b^2\right)^{k_i} \bmod a = \prod_{i=0}^{n-1} \left(\left(b^{2^i}\right)^{k_i} \bmod a\right) \bmod a \tag{13}$$

and this equation will be much efficient for computing c rather than direct calculation, while the code is shown in the Appendix [11]. Apply the equation above into above, we can find the value of c through recursion in programming,

$$p \bmod 2 = k_0$$
$$\text{if } k_0 = 1, c_1 = (1 * b) \bmod a = 2, p_1 = (p-1)/2, \text{ elseif } k_0 = 0, p_1 = p/2$$
$$b_1 = b^2 \bmod a$$
$$\vdots$$
$$p_i \bmod 2 = k_i$$
$$\text{if } k_i = 1, c_{i+1} = (c_i \cdot b_i) \bmod a, p_{i+1} = (p_i - 1)/2, \text{ elseif } k_i = 0, p_{i+1} = p_i/2$$
$$b_{i+1} = b_i^2 \bmod a$$
$$\vdots$$
$$p_{n-1} \bmod 2 = k_{n-1} = 1$$
$$c_n = c = (c_{n-1} \cdot b_{n-1}) \bmod a$$
$$c = \left(b^{k_0} \cdot b_1^{k_1} \cdot b_2^{k_2} \cdot b_3^{k_3} \ldots b_{n-1}^{k_{n-1}}\right) \bmod a$$
$$= \left(b^{k_0} \left(b^2 \bmod a\right)^{k_1} \ldots \left(\left(\ldots \left(b^2 \bmod a\right)^2 \bmod a\right)^2 \ldots\right)^2 \bmod a^{k_{n-1}}\right)$$
$$= \left(b^{k_0} \cdot b^{2k_1} \cdot b^{2^2k_2} \cdot b^{2^3k_3} \ldots b^{2^{n-1}k_{n-1}}\right) \bmod a$$
$$= \left(b^{k_0+2k_1+2^2k_2+\ldots+2^{n-1}}\right) \bmod a = b^p \bmod a$$

The corresponding code is shown in Appendix as the function "ll ModularPow(ll base, ll exponent, ll modulus)".

4.4 The function {x}

Let $\{x\}$ denotes the fractional part of x. It means that $\{x\} = x - |x|$, where $|x|$ is the integer part of x. Then, there are three useful equations that is trivally true.

$$\left\{\sum x_i\right\} = \sum \{x_i\} \tag{14}$$

$$\left\{\sum \frac{x}{y}\right\} = \sum \left\{\frac{x}{y}\right\} = \sum \left\{\frac{x \bmod y}{y}\right\} = \left\{\sum \frac{x \bmod y}{y}\right\} \tag{15}$$

$$\left\{\sum x_i\right\} = \{\ldots\{\{x_1\} + x_2\} + x_3\} + \cdots + x_i\} + \cdots\} \tag{16}$$

4.5 Discussion of the reliability of computer calculations

However, there might be some errors made in the summation of computation, like the computation will fail to add a small number to a relative large one. The reason why the error occurs is the nature of data storage in computation. The problems for the computers are that they can only save a number in a limited range and can only add or subtract the numbers having identical exponent. Therefore, when a large number is added to a relative small number, the small number will be converted to a higher exponent as same as the large number, and the most significant digit may be vanished during this computation. For instance, if there is a computer that uses base of 10 arithmetic and has 4 decimal place for mantissa and 2 places for the exponent, when adding 2 numbers 9999000 and 4, the calculation for result in computing can be shown as

$$9999000 = 0.9999 \times 10^7$$

$$4 = 0.4 \times 10^1 = 0.0000 \times 10^7$$

$$9999000 + 4 = (0.9999 + 0.0000) \times 10^7 = 0.9999 \times 10^7 = 9999000$$

While the solution of the summation should been 9999004.
In our cases, since the total length of the long double type is fixed, the incresase of integer width will lead to the decrease of fraction width, which will cause loss of data and lower precision. For example, if x = 0.385620915032679738562, then x+10 = 10.38562091503267973856. Then fractional part 0.00000000000000000002 is automatically rounded due to the limit length of storage in computer. The way to avoid such error caused by the computer is shown in the next section.

5 The implement of the BBP algorithm

The BBP formula for π:

$$\pi = \sum_{k=0}^{\infty} \frac{1}{16^k} \left(\frac{4}{8k+1} - \frac{2}{8k+4} - \frac{1}{8k+5} - \frac{1}{8k+6} \right) \tag{17}$$

Multiply two sides by 16^d and rearrange the equation,

$$\begin{aligned}16^d\pi =&4\left(16^d\sum_{k=0}^{\infty}\frac{1}{16^k(8k+1)}\right)-2\left(16^d\sum_{k=0}^{\infty}\frac{1}{16^k(8k+4)}\right)\\&-\left(16^d\sum_{k=0}^{\infty}\frac{1}{16^k(8k+5)}\right)-\left(16^d\sum_{k=0}^{\infty}\frac{1}{16^k(8k+6)}\right)\end{aligned}\tag{18}$$

Let $S_j=\sum_{k=0}^{\infty}\frac{1}{16^k(8k+j)}$, then

$$16^d\pi=4\left(16^dS_1\right)-2\left(16^dS_4\right)-\left(16^dS_5\right)-\left(16^dS_6\right)\tag{19}$$

Take the fractional part of two sides, we have

$$\left\{16^d\pi\right\}=\left\{4\left\{16^dS_1\right\}-2\left\{16^dS_4\right\}-\left\{16^dS_5\right\}-\left\{16^dS_6\right\}\right\}\tag{20}$$

Now, our task is to calculate the fractional part of 16^dS_j.
First, we can split S_j into two parts:

$$16^dS_j=\sum_{k=0}^{\infty}\frac{16^{d-k}}{(8k+j)}=\sum_{k=0}^{d}\frac{16^{d-k}}{(8k+j)}+\sum_{k=d+1}^{\infty}\frac{16^{d-k}}{(8k+j)}\tag{21}$$

It follows that

$$\left\{16^dS_j\right\}=\left\{\left\{\sum_{k=0}^{d}\frac{16^{d-k}}{8k+j}\right\}+\sum_{k=d+1}^{\infty}\frac{16^{d-k}}{8k+j}\right\}\tag{22}$$

For the first part: $\sum_{k=0}^{d}\frac{16^{d-k}}{8k+j}$, since $\left\{\sum\frac{x}{y}\right\}=\left\{\sum\frac{x \bmod y}{y}\right\}$, we have

$$\left\{\sum_{k=0}^{d}\frac{16^{d-k}}{8k+j}\right\}=\left\{\sum_{k=0}^{d}\frac{16^{d-k}\bmod 8k+j}{8k+j}\right\}$$

To compute x^n mod m, the Binary Exponentiation Algorithm is used. It's written as the function "ll ModularPow(ll base, ll exponent, ll modulus)" shown in the appendix. Basically, this function can return $[(base^{exponent})$ mod modulus$]$ efficiently. And the mathematical proof is given in Section 6.3.
In order to avoid the error caused by the limit length of storage, the Equation 16 introduced in Section 4.4 is used, that is,

$$\left\{\sum x_i\right\}=\{\ldots\{\{x_1\}+x_2\}+x_3\}+\cdots+x_i\}+\cdots\}$$

Basically, we need to take the fractional part of the sum every step we add $\frac{16^{d-k}}{8k+j}$ instead of taking the fractional part after $\sum_{k=0}^{d}\frac{16^{d-k}}{8k+j}$ is calculated. The corresponding code is

```
x += (ld)ModularPow(16,d-k,8*k+j)/(8*k+j);
x = frac(x);
```

That's a crucial improvement in the accuracy of the code.
For the second part: $\sum_{k=d+1}^{\infty} \frac{16^{d-k}}{8k+j}$, since k is always larger than d and k keeps increasing, $16^{d-k}/(8k+j)$ quickly becomes sufficiently small so that they can be ignored. Therefore, only a few terms need to be calculated and added to the sum. In the C++ code shown in appendix, only ten terms are calculated, namely,

$$\sum_{k=d+1}^{\infty} \frac{16^{d-k}}{8k+j} \approx \sum_{k=d+1}^{d+10} \frac{16^{d-k}}{8k+j}$$

In this way, the arithmetic speed increases at the cost of precision, which means less hex digits can be shown in the final result.
Combining those two parts, we have

$$\left\{16^d S_j\right\} \approx \left\{ \left\{ \sum_{k=0}^{d} \frac{16^{d-k} \bmod 8k+j}{8k+j} \right\} + \sum_{k=d+1}^{d+10} \frac{16^{d-k}}{8k+j} \right\} \tag{23}$$

Then, after calculating the value of $\left\{16^d S_j\right\}$ for j = 1,4,5,6, the fraction of π beginning at position d+1 can be found using Equation 20.
The final task is to convert the fraction into hexadecimal digits. It's written as the function "string deci2hex(ld x)" shown in the appendix. And the mathematical proof is given in Section 6.1.
The computational results of the BBP algorithm using the C++ code in Appendix are shown in the table below. After inputing the "Position", the code will return fraction, hex digits and the total run time.

Position	fraction	hex digits	Total time
10^0	0.141592654	243F6A8885	0.000173
10^1	0.352303338	5A308D3131	0.000188
10^2	0.760185039	C29B7C97C5	0.000372
10^3	0.205552819	349F1C09B0	0.00263
10^4	0.408883084	68AC8FCFB8	0.020543
10^5	0.325662698	535EA16C40	0.24638
10^6	0.151464362	26C65E52CB	2.773363
10^7	0.092519307	17AF5863EF	33.170182
10^8	0.924686485	ECB840E219	370.858805

Table 1: The computational results of the BBP algorithm

The result is the first 10 hexadecimal digits of π beginning at exact position "Position" (Position = $10^0, 10^1, ...$). It's worth noting that if the code in the main function changes from

$$\text{“}fraction = Calcu_pi(d-1);\text{”}$$

$$\text{to “}fraction = Calcu_pi(d);\text{”}$$

Position	hex digits
10^6	26C65E52CB4593
10^7	17AF5863EFED8D
10^8	ECB840E21926EC
10^9	85895585A0428B
10^{10}	921C73C6838FB2

Table 2: The known table of hexadecimal digits [1]

, the result will become the first 10 hexadecimal digits of π beginning at "Position + 1".
After testing for about fifty "Positions" and comparing to known table of hexadecimal digits, we draw the conclusion that the maximum precision of our code is 12 hexadecimal digits. It means that the hexadecimal digits begining at 13 given by the code may go wrong. And the reason lies in the limitation of the storage of numbers with fraction.
What's more, the code was run in Xcode on a Mac with a 2.7 GHz CPU. The run time may be different when run in different systems. But through observation, we can draw the conclusion that the run time is approximately proportional to the magnitude of "Position".
Note: It can be deducted that calculating the hex digit of π beginning at position 10^9 will take more than half an hour. Therefore, we stop testing at position 10^8.

6 Biography

6.1 Jonathan Michael Borwein

Born on 20 May 1951 in St. Andrews, Scotland, and died at his 65, on 2 August 2016, Jonathan Borwein was a mathematician whose area widely spreads from pure (analysis), to which our project is related, applied (optimization), computational (numerical and computational analysis) mathematics, and high performance computing. [3]
He was raised in a mathematician family. His father David Borwein, as well as his brother Peter Borwein are both mathematicians, and his co-researchers. He finished his Honours Mathematics degree as the University of Wstern Ontario in 1971 and received his PhD from Oxford in 1974. After that, he worked at Dalhousie(1974-91), Carnegie-Mellon(1980-82), Waterloo(1991-93), and SFU(1993-2016).
Jonathan Borwein won the 1993 Chauvenet Prize and Hasse Prize along with Peter Borwein, David Bailey for their contribution to finding a proof for computing one billion digits of pi.
When working at SFU, he directed the Centre for Experimental and Constructive Mathematics along with Peter Borwein, where Simon Plouffe works. They are

three of the four authors of *The quest for pi*.

6.2 Peter Benjamin Borwein

Born on 10 May 1953 in St. Anderews, Scotland, Peter Borwein is famous for his contribution to the Bailey-Borwein-Plouffe Algorithm, which is a new formula discovered by Simon Plouffe to computing pi.
He finished his Honours Mathematics degree at the same unversity as his brother in 1974, and he continued his MSc and PHD at the University of British Columbia. Then he moved to Dalhousie University, where he worked with his brother and won Chauvenet Prize and Hasse Prize.
After that he moved to SFU, joining his brother directing the Centre for the Centre for Experimental and Constructive Mathematics. He is also a co-author of *The quest for pi*

6.3 David Harold Bailey

Born in 1948, David Bailey is a mathematician and computer scientist. His research area includes High performance computing, Computational, experimental mathematics and Financial mathematics[4].
David Bailey finished his mathematics degree in Brigham Young University in 1972, and received his Ph.D. in mathematics from Stanford in 1976. During his 14 years working in NASA, he wrote many books and papers, in the field of computational and experimental mathematics and Financial mathematics, many of them are co-authored with Broweins.

6.4 Simon Plouffe and His Story

Born On 11 June 1956 in Saint-Jovite, Quebec, Simon Plouffe is a mathematician famous as the discoverer of the BBP formula.
He was a Research Associate at the Centre for Experimental and Constructive Mathematics, where he found his famous BBP algorithm. In his statement, he found the formula alone. He believed that to accept the collaboration of Peter Borwein and David H. Bailey as co-founders of that algorithm and formula is the "biggest mistake", because " they have found nothing at all...added to the group 2 months after the discovery."[5] He also claimed that Jonathan Borwein did not put an effort on this formula, but they still had his photo on the article. In his view, he stolen too much credit from his work while did nothing, and he, as the true discoverer, gained nothing, even can not get a job as professor at SFU.
His story explains the darkness behind a splendourous discovery, also shows the necessity of respecting other's work. Steal other's work is the easiest way to gain credit, but it is not honourable. No matter who he is, the stealer shall be blamed.

7 Conclusion and Discussion

In this project, our group mainly finds the value of π in hexadecimal numerical system by applying the Borwein-Bailey-Plouffe formula (BBP formula) in computation.

In the beginning, we explain and prove that the solution of the BBP formula is equal to the value of π, and also explain the PLSQ computer' s algorithm that how it is utilized for finding the integer relation with the introduction of integer relation. Afterwards, by introducing several numerical systems and methods like the hexadecimal numerical system, modulo calculation and exponential modulo bases, our group finds and explains how the BBP formula can be utilized to calculate an arbitrary hexadecimal digit of π without calculating the former digits. For the application of BBP formula for calculating π, we apply the binary exponentiation algorithm into computation and find the solution of the hexadecimal digits in different positions of π. Comparing our solutions with the solutions that have been found in previous study, we find that the time cost for computing the results will dramatically increase with the values of positions: the run time is approximately proportional to the magnitude of "Position". And we find that there are some deviation between the solutions in previous research and ours. After researching and analyzing, we find that the errors mainly are caused by the nature of computation that there are only limited room for a number to save and there may be some of the most significant digits vanished during the process of summation. The maximum precision of the code in Appendix is 12 hexadecimal digits.

Therefore, for the future study, we can try to find a methods in computing that can avoid vanishing the most significant digits and hence improve the accuracy of finding π.

8 Reference

1. David H. Bailey. "The BBP Algorithm for Pi" Lawrence Berkeley National Laboratory, Berkeley, September 17, 20. Web. July 20 2019. https://www.davidhbailey.com/dhbpapers/bbp-alg.pdf

2. "Pulling Digits out of Pi". American Mathematical Society. Web. July 29 2019. http://www.ams.org/publicoutreach/feature-column/fcarc-pi

3. Borwein, Jonathan. "Biography — The Personal Pages of Jonathan Borwein." CARMA. 14 Mar 2013. Web. 28 Jul 2019. http://carma.newcastle.edu.au/jon/index-bio.shtml.

4. Bailey, David. "David H Bailey." David H Bailey. Web. 28 Jul 2019. http://www.davidhbailey.com/.

5. S. M. Plouffe. The story behind a formula for pi. https://groups.google.com/forum/!topic/sci.math.symbolic/a3kVKVYJhgc, 2003. Web. Accessed July 29th, 2019.

6. David H. Bailey, Jonathan M. Borwein, "PSLQ: An Algorithm to Discover Integer Relations," Sept. 2009, available at https://www.davidhbailey.com/dhbpapers/pslq-comp-alg.pdf.

7. Bailey, David H. "Integer relation detection." Computing in Science & Engineering 2.1 (2000): 24.

8. Ferguson, Helaman, David Bailey, and Steve Arno. "Analysis of PSLQ, an integer relation finding algorithm." Mathematics of Computation of the American Mathematical Society 68.225 (1999): 351-369.

9. Straub, Armin. "A gentle introduction to PSLQ." (2010).

10. "Hexadecimal - Wikipedia." Wikipedia, the free encyclopedia. Wikipedia, 15 May 2019. Web. 28 Jul 2019. http://en.wikipedia.org/wiki/Hexadecimal.

11. "Modular exponentiation - Wikipedia." Wikipedia, the free encyclopedia. Wikipedia, 18 Jul 2019. Web. 28 Jul 2019. https://en.wikipedia.org/wiki/Modular_exponentiation.

12. Weisstein, Eric W. "Integer Relation." From MathWorld–A Wolfram Web Resource. http://mathworld.wolfram.com/IntegerRelation.html

13. Richmond, Michael. "Dangers of computer arithmetic." Michael Richmond's classes. Michael Richmond, Web. 29 Jul 2019. http://spiff.rit.edu/classes/phys317/lectures/dangers.html>

A Code in C++ to implement the BBP algorithm

```
#include <iostream>
#include <iomanip>
#include<string>
#include<cmath>
#include<ctime>
#define ll long long
#define ld long double
using namespace std;

//return the fractional part of x
ld frac(ld x)
{
    return x - floor(x);
}

//return (base^exponent) mod modulus
ll ModularPow(ll base, ll exponent, ll modulus)
{
    ll result = 1;
    while (exponent > 0)
    {
        if (exponent % 2 == 1)
            result = (result * base) % modulus;
        exponent /= 2;
        base = (base * base) % modulus;
    }
    return result;
}

//return {16^d Sj}
ld Calcu_Sj(ll d, int j)
{
    ld x = 0,y = 0,result = 0;
    for(ll k=0;k<=d;k++)
    {
        x += (ld)ModularPow(16,d-k,8*k+j)/(8*k+j);
        x = frac(x);
    }
    for(ll k=d+1;k<d+11;k++)
    {
        y += pow(16,d-k)/(8*k+j);
    }
    result = frac(x+y);
    return result;
```

```
}

//return the decimal number beginning at position d
ld Calcu_pi(ll d)
{
    ld fraction;
    fraction = frac(4*Calcu_Sj(d,1) - 2*Calcu_Sj(d,4)
    - Calcu_Sj(d,5) - Calcu_Sj(d,6));
    return fraction;
}

//change decimal number into hexadecimal number
string deci2hex(ld x)
{
    int k;
    ld y;
    string hex;
    for(int i=0;i<10;i++)
    {
        y = x * 16;
        k = floor(y);
        if(k <= 9)
        {
            hex += '1'+ (k-1);
        }
        else if(k <= 16)
        {
            hex += 'A' + (k-10);
        }
        x = y - k;
    }
    return hex;
}

int main()
{
    clock_t start,end;
    start=clock();
    ll d;
    ld fraction;
    string hex;
    cout << "position: ";
    cin >> d;
    fraction = Calcu_pi(d-1);
    hex = deci2hex(fraction);
    cout << "fraction: " << setprecision(14) << fraction << endl;
    cout << "hex digits: " << hex << endl;
    end = clock();
    double endtime=(double)(end-start)/CLOCKS_PER_SEC;
```

```
    cout << "Total time: " << endtime << endl;
    return 0;
}
```